

DML: DATA MANIPULATION LANGUAGE

- In SQL sono previsti tre comandi per modificare il database:
 - INSERT**: popolamento tabelle
 - DELETE**: rimozione di righe
 - UPDATE**: cambiamento di valori nelle righe delle tabelle

DELETE

- Il comando **DELETE** rimuove una o più tuple da una relazione.

```
DELETE
FROM <NOME TABELLA>
WHERE <CONDIZIONE>;
```

- Cancella da <NOME TABELLA> tutte le righe che soddisfano <CONDIZIONE>
- Se la clausola **WHERE** è omessa allora è come se ci fosse la clausola

WHERE TRUE

e vengono cancellate tutte le righe della tabella

- NB**: Il comando DELETE non rimuove la tabella
 - Il comando DROP <Nome Tabella> lo fa

Cosa succede coi vincoli di integrità referenziale?

VENI SOTTO ↓

IL COMANDO DELETE - ESEMPI

```
DELETE FROM EMPLOYEE
WHERE LNAME='Brown';
```

```
DELETE FROM EMPLOYEE
WHERE DNO IN (SELECT DNUMBER
FROM DEPARTMENT
WHERE DNAME='Research');
```

query annidata nella clausola WHERE

IL COMANDO INSERT

- Il comando **INSERT INTO** inserisce nuove righe in una relazione.

```
INSERT INTO <Nome Tabella> [ElencoAttributi]
VALUES (ListaValori);
```

oppure

```
INSERT INTO <NomeTabella> [ElencoAttributi]
SELECT ElencoAttributi
FROM Espressione;
```

IL COMANDO INSERT (2)

INSERT INTO <Nome Tabella> [ElencoAttributi]
VALUES (ListaValori);

1. Se l'elenco degli attributi viene omesso, allora si intendono riempire i campi degli attributi in **ordine di definizione della tabella**
2. Se specifico l'elenco degli attributi, allora l'elenco deve includere tutti gli attributi TOTALI (**NOT NULL**). Decidiamo noi l'ordine.
3. Gli attributi PARZIALI non presenti, assumeranno il valore NULL o il valore di DEFAULT, se questo è specificato

```
INSERT INTO EMPLOYEE (FNAME, LNAME, SSN)
VALUES ('Richard', 'Marini', '654765876');
```

```
INSERT INTO EMPLOYEE
VALUES ('Richard', 'K', 'Marini', '654765876', '30-DEC-52', '98 Oak Forest, Katy, TX',
'M', 37000, '987654321', 4);
```

IL COMANDO INSERT - ESEMPIO

- Creare una **tabella derivata (temporanea)** che ha nome, numero di impiegati e salari totali per ciascun dipartimento:

```
CREATE TABLE DEPTS_INFO (DEPT_NAME VARCHAR(15),
NO_OF_EMPS INTEGER,
TOTAL SAL INTEGER);
```

CREATE TABLE

```
INSERT INTO DEPTS_INFO (DEPT_NAME, NO_OF_EMPS, TOTAL SAL)
```

```
SELECT DNAME, COUNT(*), SUM(SALARY)
FROM DEPARTMENT, EMPLOYEE
WHERE DNUMBER=DNO
GROUP BY DNAME;
```

separazione della
nuova tabella
(nome + attributi)

join

questo è un
risultato viene
inserito nella
nuova tabella

- Eventuali aggiornamenti successivi non influenzano la tabella
Per aggiornarle, è invece necessario definire una **view**.

IL COMANDO UPDATE

- Il comando **UPDATE** permette di **modificare** valori in una relazione:

UPDATE <NomeTabella> [alias]

SET <NomeAttributo1>=<Espressione> [, <NomeAttributoN>=<EspressioneN>]

[**WHERE** <condizione>]

1. Si lavora su NomeTabella
2. Si modificano le righe che soddisfano <condizione>
3. Per ogni riga si modificano gli attributi della clausola SET con il valore calcolato dall'espressione

IL COMANDO UPDATE - ESEMPIO

```
UPDATE PROJECT
SET PLOCATION='Bellaire', DNUM=5
WHERE PNUMBER=10;
```

aumento del 10% il salario degli
impiegati del dip. 'Research'

```
UPDATE EMPLOYEE
SET SALARY=SALARY * 1.1
WHERE DNO IN (SELECT DNUMBER
FROM DEPARTMENT
WHERE DNAME='Research');
```

restituisce il numero
del dipartim. 'Research'

ESEMPIO

Magazzino (CodArticolo, Descrizione, Quantità)

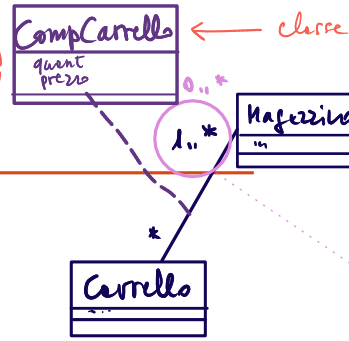
Ordine (CodOrdine, Data, PIVA, DataInvio)

CompOrdine(CodOrdine*, CodArticolo*, Quantità, Prezzo)

Carrello(CodCarrello, Data, PIVA, Completo)

CompCarrello(CodCarrello*, CodArticolo*, Quantità, Prezzo)

ESERCIZIO: fare `reverse engineering` e produrre un diagramma concettuale (ER/UML) da cui si deriva questo schema logico.



CompOrdine è enologo a CompCarrello

I casi d'uso del sistema informativo (nella nostra specifica immaginaria per questo esercizio) prevede che possa essere un carrello vuoto. Quindi il vincolo 1..* è troppo restrittivo ed è bene rimpiazzarlo con 0..* (denotato altresì con *) per ammettere carrelli vuoti.

In generale è bene fare attenzione a non imporre vincoli troppo restrittivi; è bene avere sempre come riferimento i casi d'uso.

Tabella che rappresenta la associazione tra Carrello e Magazzino (multi-a-multi)

Operazioni che devono essere fatte

Quando il carrello è completo

- Trasformo il carrello in ordine
 - Creare un nuovo ordine con la struttura del carrello
 - Rimuovere il carrello
- Aggiornare le scorte nel magazzino

Operazioni che devono essere fatte

Quando il carrello è completo

- Trasformo il carrello in ordine
 - Creare un nuovo ordine con la struttura del carrello
 - Rimuovere il carrello
- Aggiornare le scorte nel magazzino

Che operazioni di Manipolazione devo fare?

In che ordine devo fare le operazioni di manipolazione?

ESEMPIO (2)

Quando il carrello è completo

- Trasformo il carrello in ordine
 - Creare un nuovo ordine con la struttura del carrello
(INSERT in ORDINE e COMPOORDINE)
 - Rimuovere il carrello
(DELETE in CARRELLO e COMPCARRELLO)
- Aggiornare le scorte nel magazzino
(UPDATE su MAGAZZINO)

Come sempre, parte della logica di queste operazioni potrebbe essere delegata alla applicazione.

ESEMPIO (3)

Creare un nuovo ordine con la struttura del carrello (Lavoriamo con il carrello con il codice 'XYZ')

1) INSERT INTO Ordine (CodO, Data, PIVA, DataInvio)

```

(SELECT C.CodO, Data, PIVA, NULL
FROM Carrello C
WHERE C.CodO = 'XYZ')
  
```

Il risultato di questa SELECT viene inserito in Ordine

Oppure

INSERT INTO Ordine (CodO, Data, PIVA)

```

(SELECT C.CodO, Data, PIVA
FROM Carrello C
WHERE C.CodO = 'XYZ')
  
```

Omissione di DataInvio
⇒ DataInvio viene impostato a NULL

Data impostata a NULL

ESEMPIO (4)

- Creare un nuovo ordine con la struttura del carrello (Lavoriamo con il carrello con il codice 'XYZ')

2) **INSERT INTO** CompOrdine

```
(SELECT *  
FROM CompCarrello C  
WHERE C.CodC = 'XYZ')
```

ESEMPIO (5)

Cancella tutti gli elementi del carrello XYZ da CompCarrello, poi cancella il medesimo carrello XYZ da Carrello.

Lo SCHEMA
↓
È ALGIDA

2) **DELETE**

```
FROM CompCarrello C  
WHERE C.CodC = 'XYZ'
```

①

```
DELETE  
FROM Carrello C  
WHERE C.CodC = 'XYZ'
```

②

Si potrebbe fare il contrario?
Dipende da quello che abbiamo
definito
con la FK

ESEMPIO

Magazzino (CodArticolo, Descrizione, Quantità)
Ordine (CodOrdine, Data, PIVA, DataInvio)
CompOrdine (CodOrdine*, CodArticolo*, Quantità, Prezzo)
Carrello (CodCarrello, Data, PIVA, Completo)
CompCarrello (CodCarrello*, CodArticolo*, Quantità, Prezzo)

ESEMPIO (6)

- Aggiornare le scorte nel magazzino
 - Per ogni articolo nell'ordine 'XYZ' devo rimuoverne la quantità venduta nel magazzino

b) **UPDATE** Magazzino M

```
SET M.Quantità = M.Quantità - (SELECT H.Quantità  
FROM CompOrdine H  
WHERE H.CodO = 'XYZ' AND H.CodA = M.CodA)  
WHERE M.CodA IN (SELECT C.CodA  
FROM CompOrdine C  
WHERE C.CodO = 'XYZ')
```

TRANSACTION

Un **COMMIT** è la azione di consolidamento dei cambiamenti effettuati sul DB con le operazioni di modifica/inserimento/cancellazione.

Sequenza di operazioni tra due COMMIT.

ROLLBACK: ritornare allo stato del precedente COMMIT.

BEGIN TRANSACTION

INSERT...

UPDATE...

DELETE...

procedure...

COMMIT

END TRANSACTION

Questo ci consente di differire la verifica dei vincoli alla fine della sequenza di operazioni (transazione).